GIÁO TRÌNH THỰC HÀNH · LẬP TRÌNH BACKEND

CHƯƠNG 09

KIỂM THỬ, BẢO MẬT & TRIỂN KHAI HỆ THỐNG BACKEND

JEST · SUPERTEST · HELMET · SQL SERVER · DOCKER

Bài thực hành Buổi 12–14 — Hệ Liên thông



TEST → SECURE → DOCKER → DEPLOY

TIẾP NỐI PROJECT backend-quan-ly-sinh-vien TỪ CHƯƠNG 08

01 / ĐỊNH HƯỚNG

Mục tiêu, sản phẩm và điều kiện đầu vào

Chương 09 nối tiếp trực tiếp Chương 08, không tạo project mới và không thay ví dụ sang quản lý bán hàng.

Sau ba buổi thực hành, sinh viên có thể

- Testing: viết Unit Test cho Login bằng Jest/Mock, API Test bằng Supertest, đọc được PASS/FAIL.
- Security: cấu hình Helmet và Rate Limit, phát hiện rủi ro SQL Injection/XSS và lộ thông tin.
- Optimization: đo và đối chiếu truy vấn SQL Server trước/sau khi AI đề xuất tối ưu.
- Docker: đóng gói Node.js, kết nối SQL Server, kiểm chứng dữ liệu qua lần tạo lại container.
- AI-driven: phân tích → viết prompt CTCO → review code → chạy thật → lưu minh chứng.

Điều kiện đầu vào cần xác nhận

THÀNH PHẦN	YÊU CẦU GIỮ TỪ CHƯƠNG 08
Project	backend-quan-ly-sinh-vien, Express 5, TypeScript, pnpm, ESM/NodeNext
SQL Server	Sequelize 6 truy cập dữ liệu thật, không thay bằng dữ liệu RAM
API	CRUD /api/v1/sinh-vien, Login/JWT, phân quyền, Pagination/Search
Cấu trúc	AppError, Global Error Handler, Swagger YAML, Avatar/Multer
Frontend	Vite/React + Axios trong thư mục frontend/

PHẠM VI CHÍNH THỨC

Buổi 12: Automated Testing. Buổi 13: API Security và SQL Optimization. Buổi 14: Dockerization và Deployment local. GitHub/Hosting là phần mở rộng.

02 / LỘ TRÌNH

Một dự án, bốn chặng hoàn thiện Backend

Một API Login được kiểm thử, bảo vệ, đóng gói rồi kiểm chứng trên môi trường khác.

TEST Jest / Supertest › **SECURE** Helmet / Rate Limit › **DOCKER** Image / Compose › **DEPLOY** Local / Internet

Phân chia ba buổi và sản phẩm

BUỔI	LAB	ĐẦU RA CẦN CHỨNG MINH
12	Lab 1 – Automated Testing	Ít nhất 5 Unit Test; API Test; báo cáo PASS/FAIL và một lần lỗi được phân tích.
13	Lab 2 – Security + SQL	Helmet; HTTP 429 ở Login; Security Review; đo IO/TIME truy vấn SQL.
14	Lab 3 – Docker	Dockerfile và Compose; API đọc SQL thật; xác minh volume lưu dữ liệu.

Thời lượng gợi ý (90–120 phút/buổi)

HOẠT ĐỘNG	BUỔI 12	BUỔI 13	BUỔI 14
Kiểm tra dự án	10 phút	10 phút	10 phút
Chuẩn bị kỹ thuật	20 phút	15 phút	20 phút
Thực hành chính	45 phút	45 phút	50 phút
Review + kiểm thử	15 phút	20 phút	10 phút

NGUYÊN TẮC XUYÊN SUỐT

Không xóa hoặc viết lại các chức năng đã chạy đúng ở Chương 08. Chỉ bổ sung đúng file cần thiết, chạy lại pnpm build và các kiểm thử hồi quy sau mỗi chặng.

03 / HỢP ĐỒNG HỆ THỐNG

Những API và chức năng tuyệt đối không làm hỏng

Trước khi học Chương 09, ghi nhận hợp đồng API và dữ liệu đã được kiểm chứng ở các chương trước.

NHÓM	ENDPOINT / CHỨC NĂNG	KẾT QUẢ CẦN GIỮ
SinhVien	GET /api/v1/sinh-vien	200; data[] + pagination
Tìm kiếm	GET ?search=Anh	200; lọc đúng ở SQL Server
Validation	GET ?page=-1	400; response lỗi thống nhất
Đăng nhập	POST /api/auth/login	200 + JWT hoặc 401 khi sai
Profile	GET /api/auth/profile / đường dẫn thực tế	401 nếu không gửi JWT
RBAC	Student gọi DELETE SinhVien	403
Avatar	Upload và truy cập file	Giữ đúng luồng chương 07
Giao diện	React + Axios, Login, Table	Hoạt động như Chương 08

Mẫu lỗi đã học

```
{
  "success": false,
  "status": 401,
  "message": "Thông tin đăng nhập không hợp lệ",
  "data": null
}
```

LƯU Ý QUAN TRỌNG

Các đường dẫn Profile, tên trường đăng nhập và tên trường Token phải đối chiếu với mã nguồn thật của sinh viên. Không ép thay đổi response chỉ để khớp ví dụ trong giáo trình.

04 / BẢN ĐỒ PROJECT

Bổ sung file đúng vị trí, giữ project cũ

Tên file mang tính gợi ý. Nếu repository dùng tên khác, chọn file tương ứng thay vì dựng kiến trúc trùng.

```
backend-quan-ly-sinh-vien/
├── package.json        ├── pnpm-lock.yaml
├── tsconfig.json       ├── .env            # KHÔNG COMMIT
├── .env.example        ├── .gitignore
├── jest.config.mjs     # MỚI · BUỔI 12
├── Dockerfile          # MỚI · BUỔI 14
├── .dockerignore       # MỚI · BUỔI 14
├── compose.yaml        # MỚI · BUỔI 14
├── src/
│   ├── app.ts / server.ts
│   ├── config/database.ts
│   ├── routes/auth.routes.ts, sinhVien.routes.ts
│   ├── controllers/...
│   ├── services/...
│   └── middlewares/rate-limit.middleware.ts  # MỚI
├── tests/unit/login.test.ts                  # MỚI
├── tests/api/auth.api.test.ts                # MỚI
├── docs/openapi.yaml
├── docs/test-results.md, prompt-log.md
├── docs/security-report.md                 # MỚI
├── docs/sql-optimization.md, deployment-report.md
├── public/uploads/   ├── frontend/   └── sql/
```

HAI ĐIỂM PHẢI KIỂM TRA

(1) Login có thể nằm ở Controller hoặc Service: đọc file thật trước khi yêu cầu AI viết test. (2) Khi dùng Supertest có thể tách Express app khỏi app.listen(), nhưng không khởi tạo app/route thứ hai.

05 / BƯỚC 0 · CHUẨN BỊ

Khảo sát và sao lưu trạng thái Chương 08

Mọi phép thử sau này cần một trạng thái ban đầu đáng tin cậy để phát hiện lỗi hồi quy.

Thực hiện trong thư mục có package.json

```
node -v
pnpm -v
pnpm install
pnpm build
pnpm dev
```

Bảng kiểm đầu giờ

MÃ	KIỂM TRA	MONG ĐỢI
C08-01	GET danh sách SinhVien	200 + data[] + pagination
C08-02	GET ?search=Anh	200; dữ liệu được lọc
C08-03	GET ?page=-1	400 Validation
C08-04	POST Login đúng	200 + JWT
C08-05	Profile thiếu JWT	401
C08-06	Student gọi DELETE	403
C08-07	Upload Avatar	Còn hoạt động
C08-08	pnpm build	Không lỗi TypeScript

PROMPT 0 · Khảo sát, chưa viết code

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Project backend-quan-ly-sinh-vien sau Chương 08: Express 5, TS, pnpm, Sequelize 6, SQL Server, JWT, React.
T · TASK	Đọc file đã cung cấp; định vị Login, app/listen, Database, Global Error Handler, script build/run.
C · CONSTRAINT	Không tạo project mới; không tự sửa code; không đọc .env; không giả định file chưa được gửi.
O · OUTPUT	Bảng file – nhiệm vụ – giữ/sửa – lý do và kế hoạch bổ sung Jest, Helmet, Rate Limit, Docker.

ĐIỂM DỪNG 0

Chỉ bắt đầu Lab 1 khi project cũ build được và tất cả API nền cần thiết chạy đúng. Sao lưu hoặc commit trạng thái trước khi thay đổi.

06 / BUỔI 12 · LAB 1

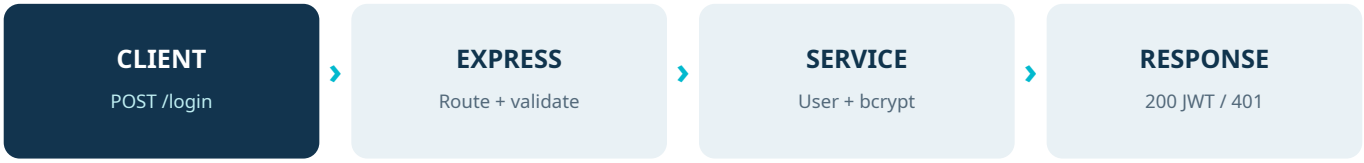
Automated Testing: mục tiêu và luồng Login

Trọng tâm là xác minh API Login: chạy được không đồng nghĩa với đúng ở tất cả tình huống.

Phân biệt các kiểu kiểm thử

KIỂU	ĐỐI TƯỢNG	VÍ DỤ TRONG PROJECT
Unit Test	Một hàm / đơn vị logic được cô lập	Logic xác thực mật khẩu với Mock
API Test	Hành vi ở giao diện HTTP	POST Login trả 200/400/401
Integration	Nhiều thành phần kết hợp	Route + middleware + service + SQL thử nghiệm

Luồng đăng nhập cần xác định trước khi viết test



- Thông tin đúng: API trả 200 và cấp Token theo hợp đồng thật.
- Sai mật khẩu / tài khoản không có: phải từ chối, không cấp Token.
- Input thiếu: bị Validation chặn trước khi xử lý nghiệp vụ.
- Lỗi Database: gửi response an toàn, không tiết lộ secret hoặc SQL stack.

PHÂN BIỆT QUAN TRỌNG

Jest là công cụ chạy bài test và assertion; Supertest gửi HTTP Request. Dùng Supertest không tự động biến một bài API Test thành Unit Test.

07 / LAB 1 · KIẾN THỨC NỀN

Từ Test Case đến mô hình Arrange → Act → Assert

Một bài test chỉ có ý nghĩa khi nêu rõ đầu vào và kết quả mong đợi trước khi viết code.

THÀNH PHẦN	CÂU HỎI CẦN TRẢ LỜI
Input	Gửi dữ liệu nào? Tài khoản có tồn tại không?
Action	Gọi hàm Login hay POST /api/auth/login?
Expected	Cần nhận trạng thái gì? Có được cấp JWT không?
Actual	Thực tế hệ thống trả kết quả gì?
PASS / FAIL	Actual có khớp Expected không?

Mô hình AAA dùng trong Jest

ARRANGE — Chuẩn bị › **ACT** — Gọi hàm / API › **ASSERT** — Đối chiếu

```
test("Sai mật khẩu phải bị từ chối", async () => {
  // Arrange: User thử nghiệm; mật khẩu nhập sai
  // Act: gọi logic đăng nhập
  // Assert: bị từ chối; không cấp JWT
});
```

CÂU HỎI NHANH

Bài test mong đợi HTTP 401. Nếu API trả 401 thì PASS; nếu API trả 200 và cấp JWT thì FAIL — lỗi logic này rất đáng quan tâm.

08 / LAB 1 · CÀI ĐẶT

Cài Jest, ts-jest và Supertest cho TypeScript

Chỉ chọn cấu hình tương thích với package.json và tsconfig.json thực tế; không đổi cả project sang CommonJS.

Các thư viện cần dùng

THƯ VIỆN	CHỨC NĂNG
jest	Test runner, assertions, báo cáo test
ts-jest	Biến đổi TypeScript cho Jest
@types/jest	Kiểu TypeScript cho Jest
supertest	Gửi HTTP Request tới Express app
@types/supertest	Khai báo kiểu TypeScript cho Supertest

Lệnh Terminal · chạy tại gốc project

```
pnpm add -D jest ts-jest @types/jest supertest @types/supertest
```

Việc sinh viên phải kiểm tra

01 Đọc package.json: có "type": "module" hay không, script hiện tại chạy thế nào?

02 Đọc tsconfig.json: module, moduleResolution và quy ước import nội bộ.

03 Tạo jest.config.mjs; tạo tsconfig.spec.json khi cấu hình test cần tách biệt.

04 Bổ sung script test và test:coverage nhưng giữ nguyên dev/build/start.

05 Chạy smoke test đơn giản, sau đó chạy lại pnpm build.

> **LƯU Ý ESM/NODENEXT**
>
> Tùy phiên bản Jest/ts-jest/Node.js, ESM có thể cần transformer, module mapper hoặc tùy chọn runtime riêng. Không sao chép cấu hình CommonJS của project khác một cách máy móc.

09 / LAB 1 · PROMPT 1

Nhờ AI cấu hình test đúng với project hiện có

Đưa đúng package.json và tsconfig.json cho AI; yêu cầu giải thích từng thay đổi và cách kiểm chứng.

PROMPT 1 · Cấu hình Jest + ESM

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Project dùng pnpm, Express 5, TypeScript, type: module, ESM/NodeNext; tôi gửi package.json và tsconfig.json.
T · TASK	Cài Jest, ts-jest, Supertest; tạo cấu hình kiểm thử phù hợp ESM và hướng dẫn chạy trên Windows.
C · CONSTRAINT	Không đổi sang CommonJS, không thay TS bằng JS, không sửa nghiệp vụ hay các script đang chạy.
O · OUTPUT	Lệnh cài, file thêm/sửa, nội dung cấu hình, một smoke test và hướng xử lý lỗi import ESM.

Kết quả cần nhìn thấy

```
> pnpm test
PASS  tests/smoke.test.ts
  ✓ Jest hoạt động trong project hiện tại

Test Suites: 1 passed
Tests:       1 passed
```

- Không tự nhận PASS: phần trên là kết quả mẫu, sinh viên phải lưu output thực tế của máy.
- Kiểm tra hồi quy: chạy pnpm build sau khi thêm cấu hình test.
- Nếu import lỗi: ghi lại package versions, thông báo lỗi đầy đủ và cấu hình hiện tại.

ĐIỂM DỪNG LAB 1 – PHẦN A

Ít nhất một bài test đơn giản chạy PASS mà không làm hỏng cấu hình ESM/NodeNext hay chức năng Backend trước đó.

10 / LAB 1 · TEST CASE

Lập bảng 5 Unit Test cho logic Login

Tạo Test Case trước, sau đó mới yêu cầu AI sinh test. Mỗi tình huống phải có Expected Result rõ ràng.

MÃ	TÌNH HUỐNG	EXPECTED RESULT
T01	Tài khoản có, mật khẩu đúng	Xác thực thành công, tạo Token
T02	Tài khoản có, mật khẩu sai	Từ chối, không tạo Token
T03	Tài khoản không tồn tại	Từ chối, không tạo Token
T04	Thiếu email/username hoặc password	Validation từ chối input
T05	Lỗi Database / logic xác thực	Xử lý an toàn, không lộ dữ liệu nội bộ

Mock làm nhiệm vụ gì?

Mock thay thế có kiểm soát truy vấn User, bcrypt, JWT hoặc phụ thuộc khác. Nhờ đó bài Unit Test kiểm tra logic mà không phụ thuộc vào SQL Server thật. Mỗi mock phải biểu diễn đúng tình huống đang kiểm tra.

```
// Pseudocode: mục tiêu là mô tả trách nhiệm của bài test.
Arrange: User giả lập, bcrypt.compare => false
Act:     gọi hàm Login đang có
Assert:  bị từ chối theo hợp đồng
Assert:  không gọi hàm tạo JWT
```

ĐIỂM CHẤM

Không chấm đạt nếu bài test giả lập cả hàm cần kiểm thử để luôn trả expected. Mock phụ thuộc, không mock chính hành vi đang muốn chứng minh.

11 / LAB 1 · MOCK

Review bài Unit Test do AI viết

Sinh viên cần đọc được phần Arrange/Act/Assert và biết Mock nào được sử dụng để cô lập logic.

PROMPT 2 · Viết 5 Unit Test cho Login

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Đây là code Service/Controller Login, interface và nơi tạo JWT thực tế tôi đã cung cấp.
T · TASK	Viết ít nhất T01–T05 bằng Jest, Mock phụ thuộc thích hợp; giải thích logic mỗi test.
C · CONSTRAINT	Không kết nối Database thật; không ghi JWT_SECRET thật; không thay chức năng Login; giữ TypeScript/ESM.
O · OUTPUT	File test đầy đủ, input/expected, mô tả Mock, lệnh chạy và giải thích describe/test/expect.

Checklist review sau khi AI trả code

- Có đủ 5 trường hợp; Expected Result trong bài test khớp bảng Test Case.
- Mock được reset/clear giữa các test để tránh phụ thuộc thứ tự.
- Test sai mật khẩu và user không tồn tại thực sự kiểm tra rằng JWT không được cấp.
- Không dùng mật khẩu thật, token thật, connection string thật trong test.
- Test FAIL được phân tích nguyên nhân chứ không sửa assertion cho dễ PASS.

> **THỬ NGHIỆM CÓ CHỦ ĐÍCH**
>
> Trên bản code thử nghiệm, giảng viên cho sinh viên làm sai một điều kiện rồi chạy lại test để quan sát FAIL; khôi phục logic đúng và chạy lại PASS.

12 / LAB 1 · SUPERTEST

Kiểm thử HTTP với Jest và Supertest

Ở lớp HTTP, cần kiểm tra cả status code, JSON và sự vắng mặt của JWT khi xác thực không hợp lệ.

Ví dụ minh họa (phải chỉnh theo API thực tế)

```
import request from "supertest";
import app from "../../src/app.js";

const response = await request(app)
  .post("/api/auth/login")
  .send({ email: "sv@example.com", password: "sai" });

expect(response.status).toBe(401);
expect(response.body.accessToken).toBeUndefined();
```

Đoạn trên là mẫu tư duy: chỉ dùng nếu project xuất Express app từ đường dẫn tương ứng và thật sự sử dụng trường email/accessToken. Nếu khác, sinh viên phải thay theo mã nguồn hiện có.

Tách app và lắng nghe cổng khi thật sự cần

```
// app.ts : tạo app, gắn middleware/routes, export app
export default app;

// server.ts : import app, kết nối SQL, rồi app.listen(...)
// Supertest gọi request(app) mà không cần mở PORT.
```

> **QUY TẮC GHÉP CODE**
>
> Không tạo Express app thứ hai hoặc mount lại cả bộ route chỉ để test. Nếu test có Database thật, phải sử dụng dữ liệu thử nghiệm và quy trình làm sạch có kiểm soát.

13 / LAB 1 · API TEST

Ma trận kiểm thử HTTP/JWT bằng Supertest

Giữ endpoint, status và format JSON từ hợp đồng đang chạy; không tự đổi mã lỗi để phù hợp ví dụ.

MÃ	TÌNH HUỐNG	HTTP MONG ĐỢI	BẰNG CHỨNG
A01	Login hợp lệ	200	Token được cấp
A02	Sai mật khẩu	401	Không có JWT
A03	User không tồn tại	401	Không có JWT
A04	Thiếu trường bắt buộc	400	Thông báo Validation
A05	GET Profile không có JWT	401	Request bị chặn
A06	GET Profile có JWT hợp lệ	200	Trả đúng dữ liệu được phép
A07	Lỗi hệ thống giả lập	500	Không lộ SQL / secret

PROMPT 3 · Supertest kiểm tra API Login

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Express 5 có /api/auth/login, JWT, middleware và Global Error Handler từ Chương 08.
T · TASK	Tạo API Test với Supertest kiểm tra A01–A07, status code, JSON và hành vi cấp Token.
C · CONSTRAINT	Không dùng tài khoản thật; không tự gọi listen; giữ hợp đồng API; cô lập DB phù hợp.
O · OUTPUT	File test, cách chạy, assertions chính và giải thích cách chuẩn bị dữ liệu test.

ĐIỂM DỪNG

Đối chiếu kết quả thực tế với bảng trên. Nếu mã nguồn cũ có hợp đồng khác, ghi rõ sự khác biệt và được giảng viên xác nhận trước khi cập nhật bài test.

14 / LAB 1 · CHẠY & BÁO CÁO

Chạy bộ test, đọc Coverage và ghi kết quả thật

Một bộ test có giá trị khi có thể chạy lại, phát hiện hồi quy và cung cấp bằng chứng kiểm chứng.

Terminal · thực hiện lần lượt

```
pnpm test
pnpm test -- --coverage
pnpm build
```

KHÁI NIỆM	CÁCH DIỄN GIẢI
PASS	Kết quả thực tế thỏa các assertion đã viết.
FAIL	Có assertion không đúng hoặc test setup gặp sự cố; cần phân tích.
Coverage	Mức code được thực thi bởi test; không chứng minh đầy đủ mọi trường hợp.
Regression	Chạy lại test cũ sau khi sửa code để tìm lỗi phát sinh.

Mẫu nhật ký docs/test-results.md

```
| ID  | Expected              | Actual          | Result |
| T02 | Sai mật khẩu: từ chối | HTTP 401; no JWT | PASS   |
| A04 | Thiếu password: 400   | ...             | ...    |

Lệnh chạy: ...
Ảnh/output Jest: ...
Lỗi đã phát hiện và cách sửa: ...
```

SẢN PHẨM KẾT THÚC BUỔI 12

Tối thiểu 5 Unit Test, API Test quan trọng, báo cáo kết quả thực tế, một tình huống FAIL được giải thích và bản build cũ vẫn hoạt động.

15 / LAB 1 · TỔNG KẾT

Checklist nộp Lab 1 và bài tập mở rộng

Hoàn thành từng mục trước khi bước sang Security; tránh mang lỗi chưa giải quyết sang Lab 2.

Checklist bắt buộc

☐	YÊU CẦU	MINH CHỨNG
☐	Jest cấu hình tương thích ESM/NodeNext	Smoke test PASS
☐	Ít nhất T01–T05 được chạy	Output Jest, file tests/unit/
☐	API Test Login/401/Validation	Output Supertest
☐	Không cấp JWT khi sai mật khẩu	Assertion rõ ràng
☐	Có một lần test FAIL đã phân tích	Nhật ký lỗi/sửa
☐	pnpm build và API cũ không hỏng	Terminal/Postman

Mở rộng: bài toán logic tính tổng giỏ hàng

Chỉ là bài Jest độc lập, không thêm chức năng giỏ hàng vào project Quản lý sinh viên. Sinh viên tự xác định quy tắc tính, làm tròn và giảm giá rồi viết test cho các tình huống:

- Giỏ hàng trống và nhiều sản phẩm.
- Giảm giá hợp lệ và giảm giá không hợp lệ.
- Dữ liệu sai kiểu, số lượng âm, giá tiền âm.

TỰ ĐÁNH GIÁ TRƯỚC BUỔI 13

Bạn có giải thích được tại sao dùng Mock? Điều gì phân biệt API Test với Unit Test? Test PASS có đồng nghĩa hệ thống chắc chắn không còn lỗi không?

16 / BUỔI 13 · LAB 2

Bảo mật API và tối ưu hóa truy vấn SQL

Bổ sung lớp bảo vệ phù hợp mà không làm hỏng JWT, Validation, RBAC hoặc Global Error Handling.

Khảo sát những lớp bảo vệ đã có

THÀNH PHẦN	CÂU HỎI REVIEW
JWT	Token tạo và xác minh ở đâu; thuật toán, thời hạn hợp lệ?
bcrypt	Password được so sánh với hash hay plaintext?
Zod	Body được validate trước Controller chưa?
RBAC	POST/PUT/DELETE có chặn role không phù hợp?
Sequelize	Raw SQL có dùng tham số an toàn không?
Error Handler	Lỗi 500 có trả stack, connection string, SQL ra client?

Các mục tiêu của Lab 2

- Cài Helmet và giải thích vai trò các response header bảo mật.
- Gắn express-rate-limit riêng cho POST Login, kiểm chứng HTTP 429.
- Review code các nguy cơ SQL Injection, XSS, secret/log và thiếu phân quyền.
- Đo truy vấn SQL Server thật qua STATISTICS IO/TIME trước và sau khi đề xuất tối ưu.

> **ĐIỂM XUẤT PHÁT**
>
> Không chấp nhận các giải pháp “bảo mật” bằng cách tắt JWT, bỏ Validation hoặc bọc mọi lỗi thành HTTP 200. Thử nghiệm chỉ được thực hiện trên hệ thống do lớp kiểm soát.

17 / LAB 2 · HELMET

Cấu hình Helmet và hạn chế gọi Login

Helmet bổ sung security headers; Rate Limit hạn chế tần suất request. Cả hai không thay thế JWT hoặc RBAC.

Cài thư viện tại thư mục Backend

```
pnpm add helmet express-rate-limit
```

Cấu hình minh họa với TypeScript

```
import helmet from "helmet";
import { rateLimit } from "express-rate-limit";

app.use(helmet());  // Gắn trước nhóm route cần bảo vệ

export const loginLimiter = rateLimit({
  windowMs: 60 * 1000,
  limit: 5,
  standardHeaders: "draft-7",
  legacyHeaders: false,
  message: {
    success: false, status: 429,
    message: "Bạn đã gửi quá nhiều yêu cầu",
    data: null
  }
});

// Gắn vào riêng route POST /login đang có:
router.post("/login", loginLimiter, loginHandler);
```

Chèn loginLimiter vào route đang có, giữ nguyên các Middleware validation/controller theo thứ tự hợp lý. Đây là cấu hình để học (5 request/60 giây), không mặc định phù hợp production.

> **GIỮ NGUYÊN HỢP ĐỒNG**
>
> Response 429 phải tuân thủ mẫu lỗi của project. Khi Helmet làm Swagger UI thay đổi hành vi CSP, xác minh và điều chỉnh chính sách ở phạm vi cần thiết thay vì tắt bảo mật toàn bộ.

18 / LAB 2 · PROMPT 4

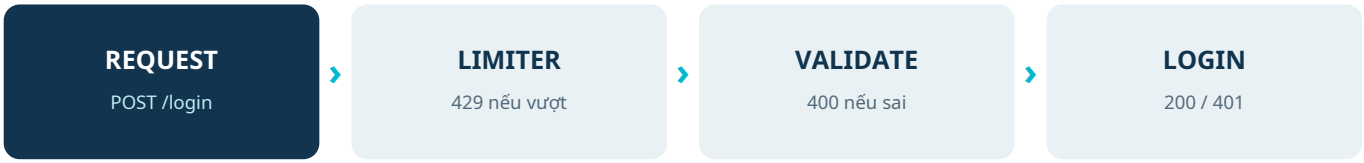
Nhờ AI tích hợp Helmet và Rate Limit đúng chỗ

Tập trung vào thay đổi tối thiểu, rõ vị trí middleware, giữ các endpoint không liên quan hoạt động.

PROMPT 4 · Helmet + Rate Limit Login

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Project Express 5, TypeScript, JWT, Swagger YAML, Global Error Handler; tôi đã gửi app/routes hiện tại.
T · TASK	Bổ sung Helmet toàn ứng dụng và limiter riêng cho POST /api/auth/login: tối đa 5 yêu cầu/60 giây, yêu cầu vượt giới hạn 429.
C · CONSTRAINT	Không xóa JWT; không sửa Controller nếu không cần; không ảnh hưởng endpoint khác; giữ response lỗi chuẩn.
O · OUTPUT	Chỉ ra file tạo/sửa, code tích hợp đúng vị trí và cách kiểm tra bằng Postman/Response Headers.

Luồng xử lý cần trình bày lại



GHI NHỚ

Cơ chế mặc định thường lưu bộ đếm trong memory của một process. Khi chạy nhiều container hoặc qua reverse proxy, cần đánh giá shared store và cấu hình nhận diện IP đúng với hạ tầng.

19 / LAB 2 · KIỂM THỬ

Mô phỏng bộ đếm 5 request và chứng minh 429

Sử dụng Postman gửi liên tiếp tới cùng API Login từ cùng định danh client trong cửa sổ giới hạn.

01 › 02 › 03 › 04 › 05 › 06

Trong ví dụ lab, các lần 1–5 đi qua limiter để Login xử lý; lần thứ 6 trong cùng cửa sổ nhận HTTP 429 Too Many Requests. Đây là minh họa quy tắc, không phải kết quả thật của server.

Ma trận kiểm thử bảo mật S01–S07

MÃ	TRƯỜNG HỢP	EXPECTED
S01	5 Login requests đầu trong cửa sổ	Được xử lý Login bình thường
S02	Request thứ 6	HTTP 429
S03	Đợi hết cửa sổ, gửi lại	Được xét hạn mức mới
S04	GET danh sách SinhVien	Không chịu giới hạn Login
S05	Kiểm tra Response Headers	Có security headers của Helmet
S06	Login sai trước khi vượt hạn mức	401, không cấp JWT
S07	Student gọi DELETE	403 theo RBAC

MINH CHỨNG PHẢI LƯU

Chụp Postman/Network cho HTTP 429 và Response Headers; ghi rõ cửa sổ thời gian, cấu hình limiter và request nào bị chặn. Không coi ảnh mã nguồn là bằng chứng đã chạy.

20 / LAB 2 · SECURITY REVIEW

Review SQL Injection, XSS và thông tin nhạy cảm

Review dựa trên file thực tế và chỉ kết luận khi có lý do kỹ thuật hoặc bằng chứng.

RỦI RO	VỊ TRÍ KIỂM TRA	HƯỚNG XỬ LÝ
SQL Injection	sequelize.query(), SQL ghép chuỗi	Bind/replacements an toàn, ORM đúng cách
XSS	React render, dangerouslySetInnerHTML	Không đưa HTML không tin cậy vào DOM
Lộ bí mật	.env, log, stack, Response 500	Không hardcode; không lộ secret ra client
Thiếu phân quyền	Routes POST/PUT/DELETE	Giữ isAuth/isAdmin
Spam Login	POST /api/auth/login	Limiter và quan sát hành vi

Đọc hiểu ví dụ nguy cơ SQL Injection

```
// KHÔNG NÊN: nối trực tiếp chuỗi từ người dùng
const sql = `SELECT * FROM SinhVien WHERE hoTen='${input}'`;

// HƯỚNG ĐÚNG: dùng ORM hoặc bind tham số,
// đối chiếu chính xác API của Sequelize đang sử dụng.
```

SQL Injection không được chữa bằng cách chỉ xóa ký tự nháy đơn. Cần tách dữ liệu đầu vào khỏi câu SQL. Với XSS, React có cơ chế escape nội dung text mặc định nhưng vẫn phải cẩn trọng với HTML tùy biến.

> **GIỚI HẠN ĐẠO ĐỨC VÀ PHẠM VI**
>
> Chỉ kiểm tra project và dữ liệu thử nghiệm do sinh viên/giảng viên sở hữu hoặc được phép thao tác. Không thử khai thác hệ thống bên ngoài.

21 / LAB 2 · PROMPT 5

Viết báo cáo Security Review dựa trên chứng cứ

Phát hiện rủi ro, nêu mức ưu tiên và chứng minh cách khắc phục bằng test hồi quy.

PROMPT 5 · Security Code Review có định hướng

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Đây là Controller, Service, Route và cấu hình Express của dự án Quản lý sinh viên đã loại bỏ secret.
T · TASK	Kiểm tra SQL Injection, XSS, secret/log, Authentication/Authorization và thứ tự Middleware.
C · CONSTRAINT	Chỉ dựa trên code được cung cấp; không tự khẳng định có lỗ hổng; không tắt JWT/Zod; không sửa schema tùy tiện.
O · OUTPUT	Bảng vị trí – rủi ro – mức ưu tiên – nguyên nhân – cách sửa – file thay đổi – cách kiểm thử.

Cấu trúc docs/security-report.md

```
# Security Review — Chapter 09
Nguồn code đã kiểm tra: [...]
1. Phạm vi và phương pháp review
2. Phát hiện (file / dòng / bằng chứng / mức độ)
3. Điều chỉnh đã thực hiện (nếu có)
4. Kết quả test trước và sau
5. Các rủi ro còn lại hoặc chưa xác minh
```

ĐIỂM DỪNG

Nêu ít nhất hai nội dung rủi ro đã kiểm tra (có thể đã được xử lý tốt trong code) và giải thích bằng chứng. Không cố tình tạo lỗ hổng để có báo cáo.

22 / LAB 2 · SQL

Phân tích truy vấn SQL có đo lường

Kết nối trực tiếp Pagination/Search của Chương 08, dùng đúng dbo.SinhVien và cấu trúc dữ liệu đang có.

Quy trình kiểm tra tối ưu hóa

ĐO TRƯỚC IO / TIME › **PHÂN TÍCH** Plan / WHERE › **ĐỀ XUẤT** AI + Review › **ĐO LẠI** So sánh

01 Chọn đúng một truy vấn đã tồn tại (ví dụ danh sách SinhVien có search, page, limit).

02 Xác định dữ liệu thử nghiệm và cách đo ổn định. Lưu kết quả đo ban đầu.

03 Đọc WHERE/ORDER BY/OFFSET/FETCH và Execution Plan nếu có.

04 Yêu cầu AI đề xuất tối ưu; không tự sửa schema hoặc thêm index chưa được duyệt.

05 Chạy lại, so sánh kết quả dữ liệu, thứ tự, logical reads và thời gian thực thi.

Ví dụ SQL thống kê minh họa (không bắt buộc)

```
SELECT maLop, COUNT(*) AS totalStudents
FROM dbo.SinhVien
WHERE hoTen LIKE N'%Anh%'
GROUP BY maLop
ORDER BY maLop ASC;
```

KHÔNG BỊA CẢI THIỆN HIỆU NĂNG

Với 5–10 dòng dữ liệu thử, thời gian chạy có thể không khác biệt đáng kể. Mục tiêu vẫn là hiểu kế hoạch thực thi, đọc số đo và phản biện đề xuất AI.

23 / LAB 2 · ĐO SQL

Đo STATISTICS IO, TIME trong SSMS

Chỉ so sánh khi hai phiên bản truy vấn bảo toàn kết quả, đặc biệt thứ tự và quy tắc phân trang.

Bước thực hành với một câu SELECT

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

-- Đặt câu SELECT cần kiểm tra tại đây.
-- Có thể xem Actual Execution Plan trong SSMS.

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Ghi số liệu từ tab Messages của SSMS và lưu hình Execution Plan nếu thực hiện. Điều kiện đo cần tương đương: dữ liệu thử nghiệm, tham số, trạng thái hệ thống và số lần chạy được mô tả rõ.

PROMPT 6 · Phân tích và tối ưu truy vấn bằng AI

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Dùng SQL Server, Sequelize 6. Tôi gửi schema dbo.SinhVien, SQL thật, số bản ghi và số đo SSMS.
T · TASK	Phân tích điểm nghẽn, đề xuất tối ưu nhưng phải giữ data và thứ tự kết quả.
C · CONSTRAINT	Không tạo bảng/cột không có; không tự thêm Index; không thay dữ liệu thật; không ghép input vào Raw SQL.
O · OUTPUT	Bảng truy vấn cũ/mới, nguyên nhân, SQL đề xuất, cách đo IO/TIME và cách so sánh kết quả.

24 / LAB 2 · KẾT QUẢ SQL

Báo cáo so sánh trước và sau tối ưu

Bảng này là mẫu để sinh viên điền số đo và ảnh chụp thật; không có số liệu mẫu giả định.

CHỈ TIÊU	TRƯỚC TỐI ƯU	SAU TỐI ƯU
Số dòng trả về	SV đo	SV đo
Kết quả và thứ tự	Ảnh/file	Ảnh/file
CPU time / elapsed time	SV đo	SV đo
Logical reads	SV đo	SV đo
Execution Plan	Ảnh SSMS	Ảnh SSMS
Kết quả có giống nhau?	Mốc chuẩn	Có / Không + giải thích

Bảng phân tích AI và kết luận

NHẬN ĐỊNH	BẰNG CHỨNG / CÁCH VIẾT
Điểm có thể chậm	Ví dụ đọc số dòng, điều kiện lọc, sắp xếp; dựa trên plan thật.
Đề xuất	Nêu chính xác thay đổi và tác động mong đợi.
Kiểm chứng	Kết quả giống nhau; số đo đã ghi và điều kiện thử.
Kết luận	Chỉ nêu cải thiện nếu dữ liệu đo hỗ trợ; nếu không, giải thích giới hạn.

SẢN PHẨM KẾT THÚC BUỔI 13

Có HTTP 429 thật, kiểm tra Helmet headers, một Security Review có căn cứ, một truy vấn SQL được đo/phân tích và kết quả test hồi quy không bị hỏng.

25 / BUỔI 14 · LAB 3

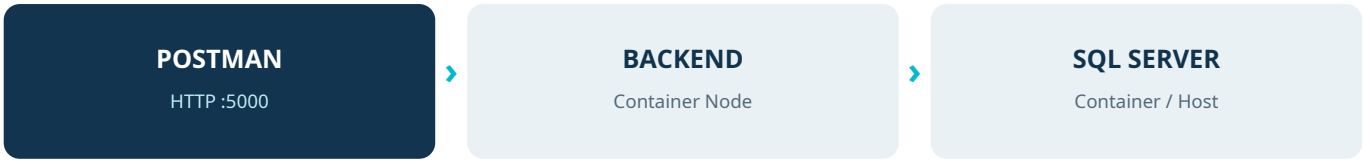
Docker hóa Backend: Image, Container và Volume

Mục tiêu là chạy được cùng project Node.js ở môi trường khác mà không cài đặt lại thủ công mọi thư viện.

Ba khái niệm không được nhầm

KHÁI NIỆM	VAI TRÒ TRONG CHƯƠNG 09
Docker Image	Bản đóng gói ứng dụng, runtime và các phụ thuộc để tạo container.
Docker Container	Tiến trình chạy từ Image, có filesystem riêng và cấu hình runtime.
Docker Compose	Mô tả/chạy nhiều service liên quan: Backend, SQL Server, network, volume.
Named Volume	Nơi lưu dữ liệu bền vững vượt vòng đời container, nếu không chủ động xóa volume.

Kiến trúc mong đợi của Lab 3



Từ Postman gửi HTTP tới Backend; Backend truy vấn SQL Server qua kết nối TCP và Sequelize. Khi dùng hai container, Backend truy cập Database qua tên service, không dùng localhost để chỉ container khác.

> **MỤC TIÊU HỌC**
>
> Image chạy được chưa đủ. Sinh viên phải chứng minh API hoạt động, kết nối đúng Database và dữ liệu không biến mất sau khi recreate container trong phương án có volume.

26 / LAB 3 · CHUẨN BỊ

Khảo sát Docker và lựa chọn phương án phù hợp

Phương án A đủ cho máy cấu hình hạn chế; Phương án B là mục tiêu Compose đầy đủ khi máy đáp ứng yêu cầu.

Terminal · kiểm tra công cụ và ứng dụng gốc

```
docker --version
docker compose version
docker info

pnpm build
pnpm start
```

Hai phương án trong cùng bài lab

PHƯƠNG ÁN	MÔ HÌNH	PHÙ HỢP
A – tối thiểu	Backend Node.js trong container; SQL Server tiếp tục chạy trên Windows host.	Máy yếu; SQL Server cũ sẵn có; ít thay đổi.
B – đầy đủ	Compose với Backend + SQL Server riêng hai container; có named volume.	Máy đủ RAM/ổ đĩa; muốn học Compose và kiểm chứng dữ liệu.

Checklist trước khi build

- Docker Desktop/Engine chạy được Linux container; kiểm tra CPU, RAM và đĩa trống.
- Image SQL Server phải hỗ trợ kiến trúc máy; chú ý yêu cầu tài nguyên riêng của Database.
- Đã biết entrypoint thực tế sau khi pnpm build, đường dẫn thư mục uploads và các biến DB.
- Sao lưu dữ liệu thử nghiệm trước khi chuyển cách kết nối SQL Server.

> **ĐIỂM DỪNG**
>
> Chọn A hay B và giải thích được DB_HOST ứng với từng trường hợp trước khi viết Dockerfile.

27 / LAB 3 · DOCKERFILE

Tạo Dockerfile multi-stage và .dockerignore

Đây là mẫu cấu trúc; phần entrypoint và layout dist/ phải đối chiếu script build của project thực tế.

Mẫu Dockerfile tham khảo · Node + pnpm + TypeScript

```
FROM node:22-bookworm-slim AS build
WORKDIR /app
RUN corepack enable
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile
COPY tsconfig*.json ./
COPY src ./src
COPY docs ./docs
# Bổ sung tài nguyên khác mà build cần dùng.
RUN pnpm build

FROM node:22-bookworm-slim AS runtime
WORKDIR /app
ENV NODE_ENV=production
RUN corepack enable
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --prod --frozen-lockfile
COPY --from=build /app/dist ./dist
# Chỉ copy tài nguyên runtime thật sự cần.
EXPOSE 5000
CMD ["node", "dist/server.js"]
```

Lưu ý: dist/server.js và Node 22 chỉ là ví dụ, không được mặc định là đúng với dự án. Nếu có asset OpenAPI, thư mục upload, Sequelize files hoặc runtime scripts khác, phải đánh giá đường dẫn tương ứng.

> **KHÔNG ĐƯA BÍ MẬT VÀO IMAGE**
>
> Dockerfile không chứa mật khẩu/JWT_SECRET. Không copy .env hoặc node_modules từ Windows. Bổ sung .dockerignore và cấu hình secret ở thời điểm chạy.

28 / LAB 3 · DOCKER BUILD

Build Image, kiểm tra output và sửa lỗi tối thiểu

Lệnh Docker nên được chạy từ thư mục root của Backend, nơi có Dockerfile và package.json.

Nội dung tối thiểu của .dockerignore

```
node_modules
.git
.env
.env.*
!.env.example
dist
coverage
*.log
Dockerfile*
compose*.yaml
frontend/node_modules
```

Đánh giá kỹ danh sách ignore theo cấu trúc thực tế; nếu file bị ignore lại cần cho runtime, phải điều chỉnh có chủ đích. Không đưa dữ liệu thật hoặc backup SQL vào build context.

PROMPT 7 · Dockerfile cho project hiện tại

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Project Express 5, TS, pnpm, ESM, SQL Server; tôi gửi package.json, tsconfig và file khởi động.
T · TASK	Tạo Dockerfile multi-stage, .dockerignore, biên dịch TS và chạy entrypoint có thật.
C · CONSTRAINT	Giữ pnpm/ESM; không copy secret/.env; không đoán dist/index.js; không thay nghiệp vụ.
O · OUTPUT	File đầy đủ, giải thích từng stage, lệnh build và cách chẩn đoán lỗi container.

Lệnh kiểm tra sau khi tạo file

```
docker build -t backend-quan-ly-sinh-vien:lab09 .
docker images
```

29 / LAB 3 · NETWORK

Kết nối SQL Server từ bên trong container

Địa chỉ localhost bên trong một container trỏ về chính container đó, không phải máy Windows hoặc service khác.

MÔI TRƯỜNG	DB_HOST THƯỜNG DÙNG	ĐIỀU KIỆN
Node.js chạy ở Windows	127.0.0.1 / hostname SQL	SQL Server mở TCP tương ứng
Backend Container → SQL trên Windows	host.docker.internal	Docker Desktop + TCP/firewall phù hợp
Backend Container → SQL Container	sqlserver	Hai service cùng Compose network

Các lớp cần kiểm tra khi kết nối thất bại

01 SQL Server Engine có chạy, TCP/IP và cổng kết nối được bật chưa?

02 Database, SQL login, quyền truy cập và phương thức mã hóa TLS đã được cấu hình đúng chưa?

03 Các biến DB_HOST/DB_PORT/DB_NAME có được đọc trong database.ts như mong đợi?

04 Firewall, network Compose và tên service đã chính xác chưa?

05 Kết nối được Database nhưng thiếu bảng? Cần khởi tạo/restore schema thử nghiệm riêng.

LƯU Ý QUAN TRỌNG

Không dùng 127.0.0.1 trong container Backend để truy cập một SQL Server container khác. Cũng không coi việc container khởi động thành công là bằng chứng kết nối Database thành công.

30 / LAB 3 · COMPOSE

Kiến trúc Compose: Backend, SQL Server, Volume

Hai service chạy trong cùng network. SQL Server giữ dữ liệu ở named volume; Avatar cũng cần nơi lưu bền vững.



Những thành phần Compose phải có

THÀNH PHẦN	CHỨC NĂNG / LƯU Ý
backend service	Build từ Dockerfile, expose/mapping cổng API.
sqlserver service	Image SQL Server phù hợp, EULA, edition và mật khẩu runtime.
Network	Backend truy cập SQL bằng tên service sqlserver.
sqlserver-data	Named volume lưu /var/opt/mssql.
uploads	Volume hoặc bind mount cho Avatar trên filesystem.
Environment	Cấp DB_HOST/DB_NAME/JWT_SECRET không hardcode.
Healthcheck	Đánh giá service thực sự sẵn sàng nhận kết nối.
Port mapping	Chỉ mở cổng cần thiết, hạn chế public SQL không cần thiết.

PHÂN BIỆT START VÀ READY

depends_on dạng đơn giản quản lý thứ tự khởi động, không đảm bảo DB đã sẵn sàng. Với condition: service_healthy cần healthcheck hợp lệ trong image đang dùng.

31 / LAB 3 · VÍ DỤ COMPOSE

Compose mẫu để AI điều chỉnh theo project thực tế

Đây là skeleton phục vụ học tập. Cấu hình DB/JWT và healthcheck phải kiểm tra trên môi trường thật trước khi chạy.

```
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      MSSQL_PID: "Developer"
      MSSQL_SA_PASSWORD: ${MSSQL_SA_PASSWORD:?missing}
    volumes:
      - sqlserver-data:/var/opt/mssql
    # Healthcheck phải chọn lệnh có trong image.

  backend:
    build: .
    environment:
      NODE_ENV: production
      PORT: "5000"
      DB_HOST: sqlserver
      DB_PORT: "1433"
      DB_NAME: ${DB_NAME:?missing}
      DB_USER: ${DB_USER:?missing}
      DB_PASSWORD: ${DB_PASSWORD:?missing}
      JWT_SECRET: ${JWT_SECRET:?missing}
    ports:
      - "127.0.0.1:5000:5000"
    volumes:
      - uploads-data:/app/public/uploads
    depends_on:
      - sqlserver

volumes:
  sqlserver-data:
  uploads-data:
```

Mẫu sử dụng depends_on cơ bản, nên chưa đảm bảo SQL ready. Sinh viên bổ sung healthcheck/đợi kết nối theo image và cấu hình thật; xác minh DB_USER có quyền và database đã tồn tại trước khi gọi API.

> **CẢNH BÁO**
>
> MSSQL_PID: Developer chỉ dành cho mục đích phát triển/kiểm thử phù hợp giấy phép, không dùng cho sản xuất thương mại. Không đưa mật khẩu thực vào compose.yaml hoặc repository.

32 / LAB 3 · PROMPT 8

Nhờ AI hoàn thiện Compose + Healthcheck

Hãy buộc AI đọc cấu hình Sequelize và Dockerfile đang có, thay vì tự đoán biến môi trường.

PROMPT 8 · Compose Backend + SQL Server

THÀNH PHẦN	NỘI DUNG PROMPT CÓ THỂ SAO CHÉP
C · CONTEXT	Dockerfile Backend đã build. Sequelize kết nối SQL Server; DB học tập là QuanLySinhVienDB (nếu đúng mã nguồn).
T · TASK	Viết Compose gồm backend + sqlserver, network, environment, named volumes, healthcheck thực thi được và hướng dẫn seed/restore.
C · CONSTRAINT	Không hardcode secret; không dùng localhost giữa container; không tự tạo schema khác; bảo toàn uploads.
O · OUTPUT	compose.yaml đầy đủ, .env.example, cách chạy, kiểm tra DB healthy và chứng minh dữ liệu sau recreate.

Cần tự xác minh trước khi dùng output AI

- Image SQL Server có thật sự chứa chương trình được gọi trong healthcheck không?
- Biến môi trường trong Compose trùng với tên mà src/config/database.ts đọc không?
- Ứng dụng có retry DB khi SQL đang khởi động chậm hay chỉ crash ngay lần đầu?
- Tên Database, các bảng và dữ liệu test đã được tạo/restore đúng quy trình chưa?
- Bản build có bỏ sót openapi.yaml, public/uploads hoặc file runtime khác không?

> **ĐIỂM DỪNG**
>
> Không tiếp tục cho đến khi docker compose config hợp lệ và đã xác định được cách khởi tạo dữ liệu thử nghiệm.

33 / LAB 3 · VẬN HÀNH

Chạy Compose và kiểm tra API end-to-end

Chạy lần lượt, quan sát logs của cả Backend và SQL Server, không kết luận dựa vào một dòng “Container started”.

Terminal · quy trình chạy

```
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs backend
docker compose logs sqlserver
```

Ma trận test Docker D01–D09

MÃ	KIỂM TRA	EXPECTED
D01	Backend container	Trạng thái chạy hợp lệ
D02	SQL Server container	DB sẵn sàng
D03	GET danh sách SinhVien	200 + dữ liệu SQL thật
D04	Login đúng	200 + JWT
D05	Login sai	401, không có JWT
D06	Gọi vượt Rate Limit	HTTP 429
D07	GET ?page=2&limit;=2	Phân trang như Chương 08
D08	Profile thiếu JWT	401
D09	Lỗi hệ thống / log	Response không lộ secret

CHỨNG CỨ THỰC HÀNH

Chụp Compose ps, log đã che secret, Postman các status quan trọng; ghi rõ dùng phương án A hay B. Với phương án A không gán kết quả SQL Server container cho một container chưa tạo.

34 / LAB 3 · LƯU BỀN VỮNG

Thử nghiệm quan trọng: dữ liệu còn sau recreate

Chỉ thực hiện trên Database test. Không sử dụng lệnh xóa named volume trong bài chứng minh persistence.

Quy trình chứng minh dữ liệu bền vững

01 Tạo một sinh viên thử nghiệm và ghi lại maSV duy nhất (hoặc chọn bản ghi test đã có).

02 Kiểm tra bằng GET API và, nếu có thể, SELECT trực tiếp ở SQL Server container.

03 Chạy docker compose down để xóa container/network nhưng giữ named volume.

04 Chạy lại docker compose up -d, đợi SQL Server và Backend sẵn sàng.

05 Gọi GET tìm cùng maSV; xác nhận dữ liệu vẫn đúng; chụp bằng chứng.

```
docker compose down
docker compose up -d
docker compose ps
# Dùng Postman/SSMS kiểm tra lại cùng maSV
```

TUYỆT ĐỐI TRÁNH

Không dùng docker compose down -v trong bài kiểm tra dữ liệu bền vững. Tùy chọn -v yêu cầu xóa cả volume mà Compose quản lý.

AVATAR CŨNG LÀ DỮ LIỆU

Nếu ứng dụng dùng public/uploads để lưu Avatar, phải chứng minh thư mục đó được mount ra volume/bind mount thích hợp; nếu không, file upload có thể mất theo container.

35 / LAB 3 · NGHIỆM THU

Mức tối thiểu và mức đầy đủ của Lab 3

Giảng viên phân biệt rõ sản phẩm theo điều kiện máy mà vẫn giữ trọng tâm kiến thức Docker.

TIÊU CHÍ	PHƯƠNG ÁN A	PHƯƠNG ÁN B
Backend trong container	Bắt buộc	Bắt buộc
SQL Server ở đâu?	Máy Windows / SQL host	Container sqlserver
Dockerfile build thành công	Bắt buộc	Bắt buộc
API truy cập dữ liệu thật	Bắt buộc	Bắt buộc
Compose hai service	Không bắt buộc	Bắt buộc
Volume SQL persistence	Kiểm chứng DB host còn dữ liệu	Kiểm chứng named volume SQL
Uploads persistence	Có phương án lưu bền vững	Có volume/mount

Hồ sơ docs/deployment-report.md

```
# Chapter 09 · Docker Report
Phương án A/B: ...
Docker Engine / SQL Server image: ...
Build image: PASS/FAIL + log rút gọn
Cấu hình network, DB_HOST: ...
Test D01–D09: ...
Dữ liệu trước/sau recreate: ...
Sự cố gặp phải và cách xử lý: ...
```

ĐIỂM DỪNG LAB 3

Image build, Backend container chạy, API kết nối SQL Server thật. Nhóm làm phương án B cần có bằng chứng dữ liệu SQL tồn tại sau recreate container (giữ volume).

36 / MỞ RỘNG · DEPLOYMENT

GitHub, Hosting và triển khai Backend lên Internet

Phần mở rộng dành cho sinh viên hoàn thành sớm hoặc học sau giờ; không thay điều kiện đạt tối thiểu của Lab 3.

Bước 1 · Đưa mã nguồn lên GitHub

```
git init
git add .
git commit -m "Complete chapter 09 labs"
git branch -M main
# Thêm remote là URL repository thật trước khi push
# git remote add origin <repository-url>
# git push -u origin main
```

- Chạy git status và review file staged; không commit .env, secret, backup chứa dữ liệu cá nhân.
- Không commit node_modules, dist, log, coverage hay Avatar riêng tư.
- Có README.md, .env.example, lệnh test/build và hướng dẫn chạy Docker.

Bước 2 · Nghiên cứu Hosting

HƯỚNG	GIÁ TRỊ THỰC HÀNH	GIỚI HẠN
Render	Triển khai Node.js Web Service từ repository.	Không mặc định có SQL Server miễn phí.
Railway	Triển khai dự án từ Docker/repository.	Tài nguyên/credit và chi phí tùy chính sách.

KHÔNG CAM KẾT MIỄN PHÍ

Dịch vụ Hosting có thể thay đổi gói, sleep, filesystem và chi phí. Trước khi triển khai phải xem điều kiện hiện hành; SQL Server cần một nơi chạy hoặc một DB kết nối được.

37 / MỞ RỘNG · PRODUCTION

Kiểm chứng API sau khi đưa lên môi trường ngoài

Một URL public mở được chưa phải triển khai thành công nếu Login, Database hoặc bảo mật vẫn bị lỗi.

Các bước kiểm tra sau deploy

01 Ghi URL public thực tế, môi trường deploy, phiên bản image/build và thời điểm kiểm thử.

02 Cấp biến môi trường và secret qua cấu hình dịch vụ; không in giá trị thật trong báo cáo.

03 Kiểm tra kết nối SQL Server, tài khoản ít quyền và các giới hạn mạng.

04 Thử Login đúng/sai, Profile cần JWT, GET danh sách và Pagination.

05 Kiểm tra security headers, Login limiter và hành vi HTTP 429 phù hợp.

06 Xem log khi lỗi khởi động; không gửi stack/secret về client.

Bảng nghiệm thu môi trường public

HẠNG MỤC	KẾT QUẢ THỰC TẾ
Public URL + HTTPS	Sinh viên ghi URL và ảnh minh chứng
GET danh sách / Login	HTTP, body mong đợi, thời điểm kiểm thử
Kết nối SQL Server	Dữ liệu thật theo môi trường test
Helmet / Rate Limit	Có chứng cứ response headers / 429
Ảnh và upload	Kiểm tra lưu trữ bền vững nếu bật chức năng

PHẠM VI CÔNG BẰNG

Hosting online là điểm cộng hoặc nội dung nâng cao. Không bắt buộc sinh viên chạy SQL Server online nếu chính sách dịch vụ, thiết bị hoặc ngân sách không đáp ứng.

38 / KIỂM THỬ TỔNG HỢP

Ma trận nghiệm thu Chương 09

Mỗi ca kiểm thử phải có Expected, Actual, PASS/FAIL và minh chứng; không chấm chỉ dựa trên mã nguồn.

NHÓM	MÃ	NỘI DUNG	BẰNG CHỨNG
Unit Testing	T01–T05	Logic Login với Mock	Jest output + file test
API Testing	A01–A07	Status/JWT/Validation	Supertest output
Security	S01–S07	Helmet + Rate Limit + RBAC	Postman / headers
Docker	D01–D09	Container + SQL + API	Compose, logs, Postman
Regression	R01–R05	Các tính năng cũ Chương 08	Postman/Frontend

Bộ 5 ca Regression đề xuất

MÃ	MỤC TIÊU
R01	CRUD SinhVien: GET/POST/PUT/DELETE theo đúng phân quyền.
R02	Pagination/Search: page, limit, search và totalItems hoạt động.
R03	Global Error Middleware: 400/404/500 trả JSON đúng hợp đồng.
R04	Avatar Upload: upload và truy cập file, giữ dữ liệu sau restart.
R05	Frontend: Login, Axios, danh sách và chuyển trang không lỗi.

Mẫu ghi docs/test-results.md

```
| ID | Expected | Actual | PASS/FAIL | Minh chứng |
|----|----------|--------|-----------|------------|
|    |          |        |           |            |
```

QUY TẮC PASS/FAIL

HTTP 200 không đủ để kết luận PASS: phải đúng nội dung JSON, điều kiện bảo mật, dữ liệu thực và logic nghiệp vụ của từng bài test.

39 / HỒ SƠ NỘP BÀI

Yêu cầu nộp bài Chương 09 qua GitHub

Nộp 01 repository GitHub tiếp nối project backend-quan-ly-sinh-vien của Chương 08; trong repository phải có mã nguồn, tài liệu và minh chứng thực hành.

CHỈ NỘP LINK REPOSITORY

Giảng viên xem trực tiếp các file và bằng chứng trong GitHub. Không chỉ nộp ảnh chụp màn hình hoặc mã nguồn chưa có báo cáo.

Các file và minh chứng bắt buộc

HẠNG MỤC	FILE / THƯ MỤC	CẦN CÓ GÌ ĐỂ KIỂM TRA
1. Mã nguồn	src/, package.json, pnpm-lock.yaml, tsconfig.json	Giữ các API Chương 08; dự án build được bằng pnpm.
2. Kiểm thử tự động	tests/unit/, tests/api/, cấu hình Jest	Có Unit Test Login với Mock và API Test với Supertest.
3. Kết quả test	docs/test-results.md	Lệnh chạy, Expected/Actual, PASS/FAIL và ảnh kết quả test thực tế.
4. Bảo mật API	docs/security-report.md + mã nguồn Helmet/Rate Limit	Ghi kiểm tra security headers; minh chứng Login bị giới hạn HTTP 429.
5. Tối ưu SQL	docs/sql-optimization.md	So sánh IO/TIME trước/sau trên dữ liệu thật; kết quả truy vấn phải đúng.
6. Docker & Compose	Dockerfile, .dockerignore, compose.yaml	Chạy container và kết nối SQL Server; có cấu hình lưu trữ phù hợp.
7. Báo cáo triển khai	docs/deployment-report.md	Ảnh/lệnh chạy API trong container; chứng minh dữ liệu còn sau recreate.
8. Nhật ký AI	docs/prompt-log.md	Prompt sử dụng, các file sửa và phần sinh viên tự review code.
9. Hướng dẫn sử dụng	README.md, .env.example	Hướng dẫn cài đặt, chạy test, chạy Compose; chỉ có giá trị cấu hình mẫu.

Cách gửi bài

1. Push đầy đủ những file trên lên GitHub; kiểm tra lại đường dẫn và ảnh trong docs/.
2. Nộp đường link repository theo kênh giảng viên quy định; nếu repository private, cần cấp quyền xem.

TUYỆT ĐỐI KHÔNG UP LÊN GITHUB

.env, mật khẩu/secret/token thật, connection string còn hiệu lực; node_modules/, dist/, bản sao lưu SQL Server chứa dữ liệu riêng tư hoặc ảnh lộ thông tin cá nhân.